

# Real-Time Model-Based Diffusion Control with Bilinear Deep Koopman Rollouts

**Abstract**—Model-Based Diffusion (MBD) solves optimal control problems by annealed, sampling-based score ascent, but it evaluates its rollout model thousands of times per control step, restricting it to platforms where a physics simulation is affordable at the control rate. We propose BK-MBD, which replaces the simulator rollout with a learned *bilinear* deep Koopman surrogate, augmented by a quantile-fitted model-error tube, at a per-step cost fixed by the lifting dimension. The bilinear form is essential: we show that whenever a configuration-dependent gain, such as a vehicle’s heading rotation or a manipulator’s Jacobian, lies between the command and the tracked output, the constant input map of a *linear* Koopman model incurs an irreducible one-step error equal to the dispersion of that gain. Closed-loop experiments on SE(2) parking and on end-effector reaching with a kinematic 7-DOF arm and a physics-simulated Franka FR3 confirm the analysis in both directions. The linear rollout fails or stalls (0/20, 0/35, and 7/35 strict on the FR3), whereas the bilinear rollout matches the true-dynamics oracle in all 90 trials and reduces the FR3 planning step from 4.1 s to 64 ms, within  $1.3\times$  of the control period. An optimizer ablation attributes the final-precision gains to the annealed schedule itself.

**Index Terms**—Koopman operator, sampling-based control, model-based diffusion, data-driven control, model learning for control.

## I. INTRODUCTION

SAMPLING-BASED optimal control has become a standard choice for robotic systems with complex nonlinear dynamics. Instead of solving a deterministic program at every step, controllers such as Model Predictive Path Integral (MPPI) control [1]–[4] and, more recently, Model-Based Diffusion (MBD) [5], [6] evaluate hundreds of sampled control sequences by forward simulation and update the nominal sequence by a cost-weighted average. MBD reinterprets this population-and-reweight primitive as one step of reverse diffusion: in this view, the weighted mean is a Monte-Carlo estimate of the score of a Gaussian-smoothed target distribution [7], [8], and repeating it along a decreasing noise schedule yields an annealed, coarse-to-fine search. Because the update requires neither gradients nor structural assumptions on the dynamics or the cost, and parallelizes naturally on modern hardware, the approach is broadly applicable. However, the *rollout model*, evaluated thousands of times at every control step, carries almost the entire computational burden and thus dictates the attainable control rate. MBD in particular inherits its rollouts from a physics simulator [5], which confines it to offline trajectory optimization or to platforms where a full simulation step is affordable at the sampling rate.

Koopman operator theory [9], [10] provides a principled way to alleviate this bottleneck by embedding the state in a space of observables, classically hand-designed [11], [12] and more recently parametrized by deep networks [13], in which the evolution is (near-)linear. A candidate trajectory can then

be propagated by a fixed cascade of matrix–vector products whose expense is set by the embedding size, *independently of the complexity of the underlying dynamics*. Linear lifted predictors dominate this literature, since, within model predictive control (MPC), only a linear predictor yields a convex program by construction [14], [15]. MPPI, in turn, has recently been paired with a learned linear deep Koopman model, with reported control performance close to true-dynamics MPPI at a fraction of its cost [16].

The linearity of the lift, however, forces the input to act through one constant matrix that does not change with the robot’s configuration. This premise fails for a large class of robots. A wheeled vehicle’s speed command displaces it along its current heading, and a manipulator’s joint-velocity command moves the end-effector through the Jacobian evaluated at its present posture. In both cases the effect of the command on the tracked output is modulated by a *configuration-dependent gain*, and this modulation cannot be represented by any constant matrix. Notably, in the experimental settings of the linear-lift controllers above, the model is never required to represent such a coupling, because the inputs (body-frame forces or accelerations, with the attitude kinematics integrated analytically) enter the learned dynamics without state dependence. We show in Sec. III that the restriction is a model-class property: the best constant surrogate is the *mean* of the gain along the visited states, leaving an irreducible one-step error equal to the gain’s dispersion. Sec. IV then shows that its closed-loop consequence is severe: the linear rollout fails outright on an analytic SE(2) testbed, stalls short of the strict tolerance on a physics-simulated manipulator, and recovers only when the coupling is removed from the learning problem by hand-supplied analytic structure, which the model-free setting withholds.

For control-affine systems, Koopman theory itself supplies the repair: the exact Koopman form carries a state-dependent input map [17], whose smallest realization is the bilinear class. Bilinear realizations have shown clear modeling and control advantages [18]–[20], although these demonstrations lie within MPC, where the bilinear term destroys the convexity that motivates the linear restriction in the first place. A sampling-based optimizer, by contrast, involves no convex program, so this restriction is no longer necessary: the bilinear model can serve directly as the rollout, at a matrix–vector cost comparable to that of the linear model. Hence, in this paper, we propose BK-MBD, a sampling-based control framework that embeds a learned *bilinear* deep Koopman rollout inside the annealed diffusion-style update of MBD. At every control step, the current output is lifted once, every candidate control sequence is evaluated in the lifted space through the bilinear recursion, and the annealed update refines the nominal sequence over a

decreasing noise schedule before the first control is executed. Since the same lifted rollout could equally drive a single-stage MPPI update, we also ask what the annealed schedule itself contributes, and answer this question by ablation in Sec. IV-E. A scalar error tube, fitted to the model's one-step residuals in the spirit of proportional error bounds for certified Koopman control [15], [21], is propagated along each candidate and added to the cost, discouraging the search from regions where the surrogate is least trustworthy. Unlike diffusion models used as *learned* motion priors [22]–[24], no trajectory distribution is learned here: the diffusion machinery serves purely as an optimizer, and the learning problem concerns the rollout dynamics.

The contributions of this paper are as follows:

- **BK-MBD: MBD at the timescale of the control period.** We embed a learned bilinear deep Koopman rollout inside the annealed MBD update, with a quantile-fitted error tube keeping the planner aware of the surrogate's trust region. The lifted rollout decouples the planning cost from the complexity of the true dynamics: it is  $64\times$  cheaper than the physics-engine oracle on the FR3, turning a 4.1 s planning step into a 64 ms one at no loss in final accuracy.
- **Controlled closed-loop validation in both directions.** Within a single MBD stack that varies only the rollout model, the linear rollout fails or stalls (0/20 and 0/35 on kinematic testbeds; 7/35 strict on a physics-simulated Franka FR3), whereas BK-MBD matches the true-dynamics oracle in all 90 trials at oracle-level final errors with no analytic model. A structure-informed linear baseline granted the analytic coupling also succeeds (90/90), which isolates the failure to the model class and delimits what the linear lift would require in deployment.
- **Isolating the contribution of the annealed schedule.** Holding the learned surrogate fixed and replacing only the update schedule, we measure what the annealed, multi-stage schedule of MBD adds over the single-stage MPPI operating point. Strict success climbs monotonically ( $25 \rightarrow 28 \rightarrow 32 \rightarrow 35$  of 35), and a Gaussian-homotopy and precision-floor analysis shows that the schedule matters exactly when the required precision approaches the noise floor of a fixed-scale sampler (Sec. IV-E).
- **Representability analysis of the linear lift.** Supporting the design, we trace the linear rollout's failure to a model-class property: the best constant input map is the mean of the configuration-dependent gain, with an irreducible one-step error equal to the gain's dispersion. The bilinear lift restores exactly the structure that the linear lift discards.

## II. PRELIMINARIES

### A. Problem Formulation

Table I summarizes the notation used throughout the paper. Consider the discrete-time system

$$s_{t+1} = f(s_t, u_t), \quad s_t \in \mathcal{X} \subset \mathbb{R}^n, \quad u_t \in \mathcal{U} = [\underline{u}, \bar{u}]^m, \quad (1)$$

TABLE I  
NOTATION AND SYMBOLS

| Symbol                                                       | Description                               |
|--------------------------------------------------------------|-------------------------------------------|
| $s \in \mathbb{R}^n, u \in \mathcal{U} \subset \mathbb{R}^m$ | system state and control input            |
| $b = b(s) \in \mathbb{R}^d$                                  | tracked output / Koopman training state   |
| $U = (u_0, \dots, u_{T-1})$                                  | control sequence over horizon $T$         |
| $J(U)$                                                       | task cost of the rolled-out sequence      |
| $z = \Psi_\theta(b) \in \mathbb{R}^r$                        | lifted state, $r \geq d$                  |
| $C$                                                          | decoder, $Cz = z_{1:d} = b$               |
| $A, B_0, B_i$                                                | lifted dynamics matrices                  |
| $M(u) = A + \sum_i u_i B_i$                                  | input-dependent transition matrix         |
| $\sigma_s, \alpha$                                           | annealing noise scale, weight temperature |
| $e_k, c_x, c_u, \beta_e$                                     | error tube and its constants / weight     |

in the *model-free, real-time* setting: the dynamics  $f$  cannot be evaluated at the control rate, whether because they are unidentified or because their simulation is too slow. The true dynamics therefore serve only as an offline *oracle* reference and cannot themselves be deployed as the controller, and model learning must proceed from interaction data  $\mathcal{D} = \{(s_i, u_i, s_i^+)\}$  alone.

Each task defines observable features  $b(s) \in \mathbb{R}^d$  used both as the Koopman training state and in the cost (unicycle:  $b = [x, y, \sin \theta, \cos \theta - 1]$ ; manipulator:  $b = [q, p_{\text{tcp}}]$ , joints and tool-center-point position). The controller operates in a receding horizon: at state  $s_t$  it seeks a control sequence  $U \in \mathbb{R}^{mT}$  minimizing

$$J(U; s_t) = \sum_{k=1}^T c(\hat{b}_k, u_{k-1}) + \phi(\hat{b}_T), \quad (2)$$

where  $\hat{b}_k$  is produced by the rollout model under study, applies the first control  $u_0$  to (1), and replans.

### B. Model-Based Diffusion: A Review

MBD [5] treats (2) as sampling from the Boltzmann target

$$p_0(U) \propto \exp(-J(U)/\alpha) \quad (3)$$

and descends an annealing ladder of Gaussian-smoothed marginals

$$p_\sigma(U) = (p_0 * \varphi_\sigma)(U) = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, \sigma^2 I)} [p_0(U - \varepsilon)], \quad (4)$$

with  $\sigma_1 > \dots > \sigma_S$ . By Tweedie's formula, the score of the smoothed target is a denoising mean shift,

$$\nabla_U \log p_\sigma(U) = \frac{\mathbb{E}[U_0 | U] - U}{\sigma^2}. \quad (5)$$

Because the smoothing kernel  $\varphi_\sigma(U - U')$  is itself the density of the proposal  $U' \sim \mathcal{N}(U, \sigma^2 I)$ , the posterior mean in (5) is a ratio of expectations under the proposal, and with  $N$  candidates  $U_k = U + \sigma \varepsilon_k, \varepsilon_k \sim \mathcal{N}(0, I)$ , projected onto  $\mathcal{U}^T$ , the self-normalized estimator is the cost-weighted mean

$$\widehat{\mathbb{E}}[U_0 | U] = \sum_{k=1}^N w_k U_k, \quad w_k = \frac{e^{-(J(U_k) - J_{\min})/\alpha}}{\sum_j e^{-(J(U_j) - J_{\min})/\alpha}}. \quad (6)$$

Under the box projection the candidates are no longer exactly Gaussian about  $U$ , so (6) estimates the score of the correspondingly projected smoothing rather than (4) itself; every

compared method shares the identical projection. The update at stage  $s$  is a preconditioned score step with optional Langevin noise, *with the step size tied to the noise level*:

$$U \leftarrow U + \eta_s \nabla_U \widehat{\log p_{\sigma_s}}(U) + \sqrt{2\eta_s} \xi, \quad \eta_s = \eta \sigma_s^2. \quad (7)$$

With  $\eta = 1$  and no Langevin noise, (7) reduces exactly to the weighted-mean update  $U \leftarrow \sum_k w_k U_k$ .

### III. PROPOSED METHOD

#### A. Deep Koopman Lifting and Rollout Models

All learned rollout models share a deep lift with an exact-decoder convention. With  $d = \dim b$ ,

$$z = \Psi_\theta(b) = \begin{bmatrix} b \\ \psi_\theta(\tau(b)) \end{bmatrix} \in \mathbb{R}^r, \quad Cz = z_{1:d} = b, \quad (8)$$

where  $\psi_\theta$  is a small tanh network and  $\tau$  a fixed task input map (unicycle:  $\tau = \text{id}$ ,  $r = 8$ ; FR3:  $\tau(b) = [\sin q, \cos q]$ ,  $r = 20$ ). Slicing the decoder avoids learning a fragile reconstruction and ties the first  $d$  lifted coordinates to the physical output. Two lifted dynamics classes are compared:

$$\textbf{linear (DK):} \quad z^+ = Az + Bu, \quad (9)$$

$$\textbf{bilinear (BK):} \quad z^+ = Az + B_0 u + \sum_{i=1}^m u_i B_i z, \quad (10)$$

where, collecting the state-multiplying terms into  $M(u) = A + \sum_i u_i B_i$ , one step of (10) reads  $z^+ = M(u)z + B_0 u$ : the transition matrix acquires an input dependence, while each step still consists of a small, fixed number of matrix–vector products. For control-affine systems the exact Koopman form carries a state-dependent input map [17]; the bilinear class is its smallest realization, restricting that dependence to be linear in the lifted state. The two classes differ in their input tangent maps: for (10),  $\partial z^+ / \partial u_i = B_0 e_i + B_i z$  varies with the lifted state through  $B_i z$ , whereas the tangent map of the linear form is the fixed matrix  $B$ . At planning time the current output is lifted once,  $z_0 = \Psi_\theta(b(s_t))$ , and every candidate is propagated entirely in the lifted space.

#### B. Why a Constant Input Map Cannot Represent the Coupling

Suppose the increment of the tracked output is first-order in the input,  $\Delta y = \Phi(s)u \Delta t + O(\Delta t^2)$ , where the gain  $\Phi(s)$  varies across configurations (the heading rotation for the unicycle; the end-effector rows of the Jacobian  $J(q)$  for the manipulator). A linear lift routes the input to the output exclusively through constant matrices:  $CB$  at one step, and the Markov parameters  $CA^k B$  at every lag. Since none of these matrices depends on the state, the full input–output response of (9) is linear time-invariant, however expressive the lifted drift may be.

**Proposition 1** (Irreducible error of a constant gain). *Let inputs satisfy  $\mathbb{E}[uu^\top] = \lambda I$  independently of the state  $s \sim \rho$ . Then*

$$\arg \min_X \mathbb{E}_{s,u} \|(\Phi(s) - X)u\|^2 = \mathbb{E}_{s \sim \rho} [\Phi(s)], \quad (11)$$

*and the minimum equals  $\lambda \mathbb{E}_{s \sim \rho} \|\Phi(s) - \mathbb{E}[\Phi]\|_F^2$ .*

*Proof.*  $\mathbb{E}_u \|(\Phi - X)u\|^2 = \text{tr}((\Phi - X) \mathbb{E}[uu^\top] (\Phi - X)^\top) = \lambda \|\Phi(s) - X\|_F^2$ ; minimizing its expectation over  $s$ , a quadratic in  $X$ , gives the mean, with the variance as the residual.  $\square$

The best constant surrogate is the *mean gain*, and the irreducible error equals the *dispersion of the gain along the visited distribution*—a model-class property that no amount of training can remove. The unicycle is the extreme case: over a parking maneuver the heading traverses the entire circle, so  $\mathbb{E}[(\cos \theta, \sin \theta)] \approx 0$  and the best constant input channel for the speed command is *zero*—the model discards the actuator. The bilinear class instead realizes the state-dependent gain  $u \mapsto C(\sum_i u_i B_i z)$ , whose columns  $CB_i z$  are linear in  $z$ . The heading rotation is therefore exactly expressible, since  $\sin \theta$  and  $\cos \theta$  are (affine) coordinates of  $z$ , and the manipulator Jacobian, whose entries are multilinear in  $\{\sin q_i, \cos q_i\}$  across joints, is expressible once the learned features supply the cross-products. We emphasize that Proposition 1 is a representability statement about the one-step, first-order input channel under the stated excitation, not a closed-loop theorem. How this representability gap is amplified into closed-loop failure once an optimizer selects controls through the model is traced empirically in Sec. IV-D.

#### C. The Structure-Informed Alternative and Its Cost

Proposition 1 indicates that the only repair available to the linear class is to remove the coupling from the learning problem. We formalize this as the DK-MBD-SPLIT baseline, which mirrors the experimental structure of linear-lift controllers [14], [16]. For the unicycle, a linear map  $G$  is fitted by least squares from the input to the *body-frame* displacement,  $[\Delta p_{\text{body}}; \Delta \theta] = Gu$  with  $\Delta p_{\text{body}} = R(-\theta_k)(p_{k+1} - p_k)$ , and rollouts compose the prediction analytically with the current heading (an  $SE(2)$  composition). For the manipulator, a linear deep Koopman model is trained on the joint state only, and the analytic forward kinematics is applied to the decoded joints when evaluating the cost. In both cases the input then enters the *learned* part state-independently, and Proposition 1 no longer binds. DK-MBD-SPLIT is the fair representative of what linear-lift control actually deploys, and its requirement is precisely the analytic rotation or forward kinematics that the model-free setting withholds. The bilinear rollout (10) needs neither.

#### D. Multi-Step Training

The planner consumes  $T$ -step rollouts, so the surrogate must remain accurate over a horizon rather than at a single transition. Models fitted by plain one-step regression drift away from the data distribution before the planning horizon is reached. All learned models are therefore trained on random-rollout snippets by the same  $H$ -step loss ( $H = 15$ ),

$$\mathcal{L} = \frac{1}{H} \sum_{h=1}^H \left[ \|C\hat{z}_h - b_h\|^2 + \gamma \|\hat{z}_h - \Psi_\theta(b_h)^{\text{sg}}\|^2 \right], \quad (12)$$

with  $\hat{z}_0 = \Psi_\theta(b_0)$ ,  $\hat{z}_{h+1}$  given by (9) or (10),  $\gamma = 0.1$ , and stop-gradient (sg) on the latent target. On the 7-DOF manipulator the  $m$  per-channel matrices make the bilinear

recursion prone to instability during training. Setting  $B_i = 0$  at initialization resolves this issue, because the optimization then begins at the linear model and introduces bilinear structure only where the data requires it. Datasets are generated once and shared across every learned model and training seed, so that the compared classes differ in nothing but the lifted dynamics (9) vs. (10).

### E. Model-Error Tube

To make the planner aware of where the surrogate can be trusted, the one-step lifted residuals on the training set,  $r_k = \Psi_\theta(b_{k+1}) - (M(u_k)\Psi_\theta(b_k) + B_0 u_k)$ , are covered by a proportional bound  $\|r_k\| \leq c_x \|z_k\| + c_u \|u_k\|$  fitted at quantile coverage. A least-squares fit of  $(\hat{c}_x, \hat{c}_u)$  to the residual norms is rescaled by the empirical  $(1 - \delta)$ -quantile of the coverage ratios,

$$s_\delta = Q_{1-\delta} \left[ \frac{\|r_k\|}{\hat{c}_x \|z_k\| + \hat{c}_u \|u_k\|} \right], \quad (13)$$

$$(c_x, c_u) = \max(s_\delta, 1) (\hat{c}_x, \hat{c}_u),$$

with  $\delta = 10^{-3}$ . Quantile coverage, rather than a worst-case fit over every residual, keeps the constants proportional to the typical model error. Sec. IV-F quantifies how a worst-case fit lets a few rest-to-motion transients dominate the planning cost. During planning a scalar tube is propagated along each candidate with the precomputed norm bound  $\bar{m}(u) = \|A\|_2 + \sum_i |u_i| \|B_i\|_2 \geq \|M(u)\|_2$ :

$$e_{k+1} = (\bar{m}(u_k) + c_x) e_k + c_x \|\hat{z}_k\| + c_u \|u_k\|, \quad e_0 = 0. \quad (14)$$

On any segment where the fitted bound holds, an induction over the exact error dynamics  $\delta_{k+1} = M(u_k)\delta_k + r_k$  of the bilinear rollout shows that the recursion dominates the true-vs-predicted lifted error:  $e_k \geq \|\delta_k\|$  with  $\delta_k = \Psi_\theta(b_k) - \hat{z}_k$ . The planner then minimizes the tube-augmented cost

$$\tilde{J}(U) = J(U) + \beta_e \sum_k e_k, \quad (15)$$

where  $\beta_e$  scales with the task cost magnitude; switching the penalty off ( $\beta_e = 0$ ) leaves the plain BK-MBD planner. The constants  $(c_x, c_u)$  are coverage estimates fitted from data, and we attach no identification-theoretic certificate to them.

In summary, a BK-MBD control step proceeds as follows. The measured output is encoded once via (8). Each annealing stage  $s = 1, \dots, S$  then draws  $N$  candidate sequences, scores them by running (10) in the lifted space while accumulating (14), forms the weights (6) on the tube-augmented cost (15), and applies (7) to the nominal sequence. Finally the first control is executed, and the sequence is shifted by one step to warm-start the next replan.

## IV. EXPERIMENTS

We evaluate the framework on two main testbeds, both first-order coupled in the sense of Sec. III-B: an SE(2) unicycle parking task, where the coupling is exact, and end-effector reaching with a Franka Research 3 (FR3) manipulator simulated with full rigid-body physics in MuJoCo. On the FR3, the joint-velocity servo realizes the first-order interface only

up to lag and load. A third testbed, a kinematic 7-DOF arm, carries the optimizer study of Sec. IV-E. The experiments are organized to support the claims of Sec. I: the failure of the full-lift linear rollout and the oracle-level performance of the bilinear rollout (Secs. IV-B–IV-C), the mechanism by which a modest open-loop gap becomes that closed-loop failure once an optimizer sits on top of the model (Sec. IV-D), the contribution of the annealed schedule over a single-stage sampler (Sec. IV-E), the rollout-cost decoupling together with an empirical validation of the tube (Sec. IV-F), and finally the boundary of the regime in which the bilinear class is needed at all (Sec. IV-G).

### A. Protocol and Compared Methods

All four planners run inside one implementation of the MBD optimizer of Sec. II-B. The *only* component that differs across methods is the rollout model that maps a candidate control sequence to a cost. MBD-TRUE rolls out the true dynamics (the analytic model on the unicycle, the MuJoCo simulator on the FR3) and serves as the oracle upper bound. DK-MBD rolls out the full-lift linear deep Koopman model (9). DK-MBD-SPLIT is the structure-informed control of Sec. III-C: it is granted the analytic coupling (the SE(2) group action on the unicycle, the forward kinematics on the FR3) and learns only the input-affine remainder. BK-MBD rolls out the bilinear model (10) with the tube cost (15). The optimizer uses  $S = 5$  annealing stages, temperature  $\alpha = 0.4$ , and step scale  $\eta = 1$  (the deterministic weighted-mean limit of (7)); per task we fix the sample count, noise schedule, horizon, and cost across all methods (unicycle:  $N = 768$ ,  $\sigma : 0.9 \rightarrow 0.2$ ,  $T = 40$ ; FR3:  $N = 800$ ,  $\sigma : 1.2 \rightarrow 0.3$ ,  $T = 15$ ).

The learned models follow one training recipe (Sec. III-D). For each of five seeds, the linear, bilinear, and split models are trained on the same dataset of 6,000 random-excitation snippets of length 15, so that every learned-model comparison shares data, architecture, epochs, and optimizer, and differs only in the lifted dynamics class. Every (task, seed) pairing then runs all closed-loop cases from identical initial sequences, giving  $5 \times 4 = 20$  trials on the unicycle and  $5 \times 7 = 35$  trials on the FR3 (and likewise 35 on the kinematic arm of Sec. IV-E). We report the task success rate, the strict success rate at a tighter tolerance, the final error, and the planning time per control step.

### B. Unicycle Parking

The unicycle state is  $(x, y, \theta)$  with speed and turn-rate inputs, integrated by RK4 at  $\Delta t = 50$  ms. The task is to park at the origin from four start poses within 180 steps, with success defined as a final position error below 0.3 m (strict: 0.08 m position and 0.2 rad heading). The base features used for lifting and cost are  $[x, y, \sin \theta, \cos \theta - 1]$ , and the coupling  $R(\theta)[v, 0]^\top$  is exactly the configuration-dependent gain of Proposition 1.

DK-MBD does not merely park imprecisely but diverges, ending with a mean final error of 2.38 m on a task whose starts are 2–3 m from the goal. Its constant input map commits to one average heading direction (Proposition 1), and the

TABLE II  
UNICYCLE PARKING, 5 SEEDS  $\times$  4 CASES.

| Method        | Park  | Strict | Final err. [m]    | ms/step |
|---------------|-------|--------|-------------------|---------|
| MBD-TRUE      | 20/20 | 20/20  | $0.070 \pm 0.008$ | 81      |
| DK-MBD        | 0/20  | 0/20   | $2.383 \pm 1.706$ | 43      |
| DK-MBD-SPLIT  | 20/20 | 20/20  | $0.066 \pm 0.010$ | 58      |
| BK-MBD (ours) | 20/20 | 18/20  | $0.071 \pm 0.026$ | 95      |

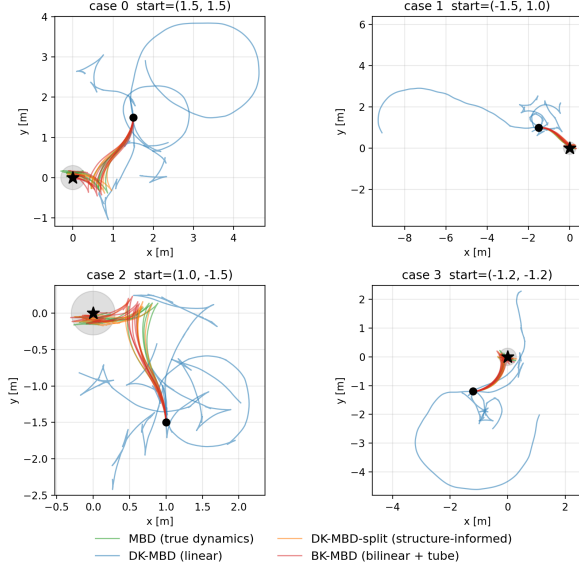


Fig. 1. Unicycle parking trajectories for all methods over five seeds (gray disk: 0.3m park radius). DK-MBD (blue) veers away in every trial, while DK-MBD-SPLIT (orange) and BK-MBD (red) are indistinguishable from the oracle (green).

annealed optimizer then faithfully optimizes the wrong model. Both control conditions behave as predicted (Table II, Fig. 1): granting the linear model the SE(2) structure (DK-MBD-SPLIT) or replacing the model class by the bilinear lift (BK-MBD) restores oracle-level parking, with final errors within one standard deviation of MBD-TRUE.

### C. Franka FR3 Reaching in Physics Simulation

The second testbed is a Franka Research 3 manipulator with attached Franka Hand, simulated in MuJoCo with full rigid-body dynamics. The robot is controlled through a gravity-compensated joint-velocity servo interface, the same abstraction exposed by the vendor controller on the physical robot, at a control period of 50 ms spanning 25 physics substeps. The surrogate must therefore absorb servo lag and the inertial load of the hand rather than an exact integrator. The control point is the tool center point (TCP) between the two fingertips, and the Koopman state is  $b = [q, p_{\text{tcp}}]$  with measured joint positions. At every control step the planner searches a 105-dimensional decision space ( $m = 7$  joint velocities over  $T = 15$  steps). Training data are 6,000 coherent random joint-velocity snippets rolled through the simulator; seven reach targets are attempted for up to 120 steps (6 s), with success below 5 cm and strict success below 2.5 cm. The oracle plans through the simulator itself with threaded batch rollouts.

TABLE III  
FR3 REACHING IN MUJoCo, 5 SEEDS  $\times$  7 TARGETS.

| Method        | Reach | Strict | Final err. [m]    | ms/step |
|---------------|-------|--------|-------------------|---------|
| MBD-TRUE      | 35/35 | 35/35  | $0.023 \pm 0.002$ | 4094    |
| DK-MBD        | 13/35 | 7/35   | $0.054 \pm 0.022$ | 37      |
| DK-MBD-SPLIT  | 35/35 | 35/35  | $0.023 \pm 0.001$ | 56      |
| BK-MBD (ours) | 35/35 | 35/35  | $0.023 \pm 0.002$ | 64      |

TABLE IV  
RULING OUT CAPACITY AND BLACK-BOX EXPLANATIONS ON THE FR3; HELD-OUT OPEN-LOOP TCP ERROR (TRAINING SEED 0), ROLLOUT COST OF AN 800-CANDIDATE BATCH, AND CLOSED-LOOP STRICT SUCCESS UNDER THE IDENTICAL PLANNER (5 SEEDS  $\times$  7 TARGETS; LINEAR ROW FROM TABLE III).

| Model           | $r$ | $H=15$ [m] | ms  | Strict       |
|-----------------|-----|------------|-----|--------------|
| Linear          | 20  | 0.336      | 0.3 | 7/35         |
| Linear (large)  | 57  | 0.336      | 0.6 | —            |
| MLP dynamics    | —   | 0.236      | 3.0 | 26/35        |
| Bilinear (ours) | 20  | 0.252      | 1.1 | <b>35/35</b> |

Table III shows that the pattern transfers to full physics. BK-MBD again matches the oracle exactly ( $0.023 \pm 0.002$  vs.  $0.023 \pm 0.002$  m, all trials strict) and traces the same direct TCP paths. DK-MBD, in contrast, is unreliable: it wanders across the workspace, reaching the strict tolerance in only 7/35 trials and exceeding the loose one in 22/35. The planning-time column carries the practical significance of these results. Rolling out the physics engine costs the oracle 4.1 s per 50 ms control step, two orders of magnitude away from real time even with threaded rollouts. BK-MBD plans in 64 ms, a  $64\times$  reduction that brings the same annealed optimizer to within  $1.3\times$  of the control rate while losing nothing in final accuracy.

Two alternative explanations for the linear rollout’s failure remain to be examined: insufficient capacity, and the possibility that a generic nonlinear dynamics model would suffice. Table IV addresses both. Regarding capacity, the enlarged linear lift at  $r = 57$  spends more dynamics parameters, and a higher per-step FLOP count, than the bilinear model at  $r = 20$ , yet its held-out TCP error does not move to three decimals; the limitation is the class-level ceiling of Proposition 1, not a parameter shortage. Regarding the second explanation, an MLP dynamics model given the same features, multi-step loss, and budget does capture the coupling, and its open-loop accuracy even slightly exceeds the bilinear model’s. In closed loop, however, it stalls at the step limit 2.9–4.7 cm from the target and reaches the strict tolerance in only 26/35 trials against 35/35 for the bilinear rollout—a second instance of open-loop accuracy failing to certify closed-loop precision (Sec. IV-D). It also incurs a  $2.7\times$  higher rollout cost and sacrifices the lifted-space linearity that the tube recursion (14) exploits. Among these candidates, only the bilinear lift combines nonlinear-class accuracy, matrix–vector rollout cost, tube-compatible structure, and oracle-level closed-loop precision.

### D. From Open-Loop Error to Closed-Loop Failure

The closed-loop results above raise a question about the mechanism: does the linear lift fail because it is a much worse *predictor*, or because of how an *optimizer* uses it? Because MBD interrogates the rollout model  $S \times N$  times

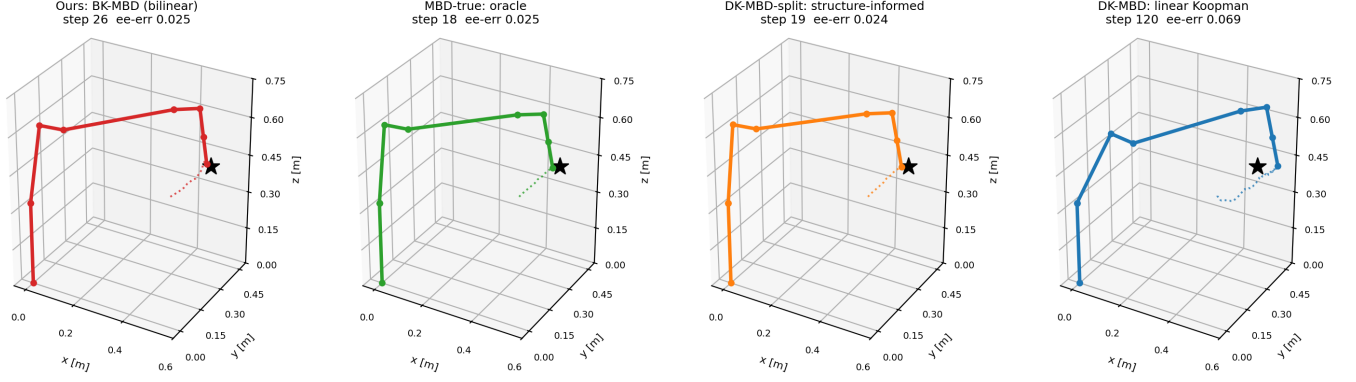


Fig. 2. Final FR3 configuration (bold kinematic chain) and TCP path (dotted) on the seventh reach target for each method. BK-MBD, the oracle, and DK-MBD-SPLIT bring the tool to the target ( $\star$ ) in 19–26 steps; DK-MBD leaves the arm short of the goal at the step limit (6.9 cm).

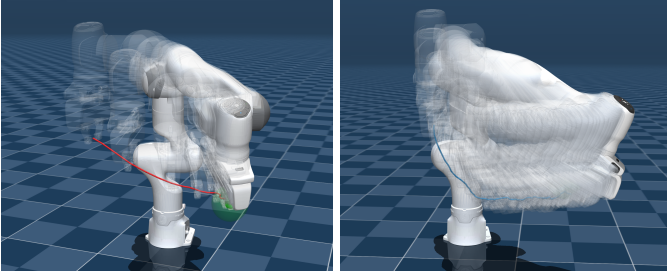


Fig. 3. Executed closed-loop FR3 motions on a representative reach, rendered as afterimages (translucent snapshots along the trajectory). BK-MBD (left) sweeps directly to the target (green sphere; TCP trace in red), whereas DK-MBD (right) wanders across the workspace without settling on the target (blue trace).

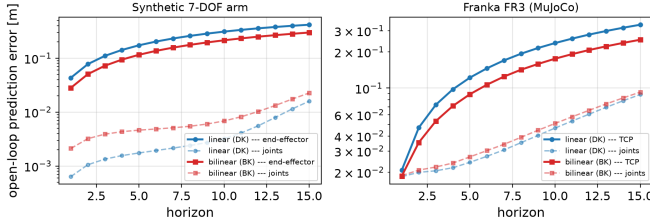


Fig. 4. Open-loop prediction error vs. horizon for the trained FR3 checkpoints used in closed loop (mean  $\pm$  std over five seeds; held-out data, log scale). The classes coincide on the joint rows (dashed); on the TCP rows (solid) the linear model is only  $1.33\times$  worse at the planning horizon  $H = 15$ —yet reaches the strict tolerance in only 7/35 closed-loop trials against 35/35 for the bilinear model.

per control step and executes whatever the model scores best, the distinction matters, and it can be read off the open-loop error of the very checkpoints used in closed loop. Figure 4 shows the open-loop prediction error of the FR3 checkpoints against the rollout horizon on held-out data, averaged over the five training seeds and decomposed into the output rows that carry the configuration-dependent coupling (TCP) and the joint rows, where the input enters state-independently.

Two observations localize the failure. First, the penalty is exactly where Proposition 1 predicts: on the joint rows the two classes are indistinguishable, while on the coupled rows the linear model carries a persistent excess error. Second, and

TABLE V  
OPTIMIZER ABLATION ON THE KINEMATIC ARM: IDENTICAL BILINEAR CHECKPOINTS, COST, AND TUBE; ONLY THE UPDATE SCHEDULE VARIES (5 SEEDS  $\times$  7 TARGETS). (A) IS THE MPPI OPERATING POINT; (D) IS THE FULL ANNEAL USED BY BK-MBD.

| Schedule                                           | Reach        | Strict       | Final err. [m]                      | ms |
|----------------------------------------------------|--------------|--------------|-------------------------------------|----|
| (a) $1 \times 800$ , $\sigma \equiv 1.2$           | 32/35        | 25/35        | $0.029 \pm 0.013$                   | 18 |
| (b) $1 \times 4000$ , $\sigma \equiv 1.2$          | 32/35        | 28/35        | $0.028 \pm 0.008$                   | 24 |
| (c) $5 \times 800$ , $\sigma \equiv 1.2$           | 34/35        | 32/35        | $0.027 \pm 0.017$                   | 71 |
| (d) $5 \times 800$ , $\sigma: 1.2 \rightarrow 0.3$ | <b>35/35</b> | <b>35/35</b> | <b><math>0.024 \pm 0.001</math></b> | 69 |

more consequential, the open-loop gap is *modest*— $1.33\times$  at the planning horizon—because the Jacobian along reaching motions has a non-zero mean, which the constant input map of Proposition 1 captures. The irreducible part is only the *dispersion* around that mean. In closed loop, this modest gap becomes 7/35 strict versus 35/35. The amplification is a property of optimizing through the model rather than predicting with it. The residual left by the mean gain is a *structural bias* that always points the same way for a given configuration; the annealed search selects the candidates whose predicted progress exploits this bias, and receding-horizon replanning re-commits to the biased direction at every step. Fig. 2 and Fig. 3 visualize this end state: DK-MBD leaves the arm short of the goal when the step limit expires, while every other method brings the tool to the target. The unicycle lies at the opposite end of the same spectrum: a parking maneuver drives the heading around the entire circle, leaving a near-zero mean gain, so the one-step open-loop gap is already  $40\times$  ( $0.082$  vs.  $0.002$  RMSE) and the closed loop diverges outright. Open-loop error therefore under-reports the model-class deficit in proportion to how benign the visited gain distribution is. A rollout surrogate for a sampling-based optimizer must consequently be judged in closed loop, on the coupled output rows, rather than by aggregate prediction scores.

### E. Contribution of the Annealed Schedule

The optimizer itself is the remaining free variable: does the annealed, multi-stage schedule of MBD contribute anything that a single-stage sampler—the MPPI operating point—would not achieve with the same surrogate? We answer on a third



testbed on which the question is sharpest: a kinematic 7-DOF arm reaching end-effector targets under joint-velocity inputs, with the same training recipe and protocol (5 seeds  $\times$  7 targets, success below 5 cm, strict below 2.5 cm). Its main comparison mirrors the FR3 pattern: the linear rollout fails outright (0/35, final error  $0.73 \pm 0.40$  m), while BK-MBD matches the oracle (35/35, 0.024 vs. 0.023 m). Holding the trained bilinear checkpoints, task, cost, and tube fixed, we then vary *only* the update schedule (Table V).

Strict success climbs monotonically as optimizer structure is added ( $25 \rightarrow 28 \rightarrow 32 \rightarrow 35$ ), but only the full anneal reaches 35/35, at the same rollout budget as the fixed-noise five-stage variant (c). Five times more samples in a single stage (b) yield almost no improvement. Two mechanisms explain the ordering. First, with a learned surrogate the evaluated cost is  $\hat{J} = J + e$ , where the model-error term  $e$  inherits fine-scale structure from error accumulation along rollouts. Each annealing stage descends a Gaussian-smoothed landscape in which the Fourier components of  $e$  are attenuated by  $e^{-\sigma^2 \|\omega\|^2/2}$ : early (large- $\sigma$ ) stages select the basin on a landscape whose spurious model-induced minima are suppressed, while late stages refine where local bias is small. A single fixed scale must serve both roles at once. Second, a fixed scale has a *precision floor*: the weighted-mean estimator carries Monte-Carlo noise of order  $\sigma/\sqrt{N_{\text{eff}}}$  per axis ( $N_{\text{eff}} = 1/\sum_k w_k^2$ ). In receding horizon the control is executed after a single update, so this noise enters the plant directly. The failure signature confirms the mechanism: seven of the eight fixed-noise failures across (a)–(c) are marginal stalls at 0.050–0.079 m against the 0.05 m threshold—never divergence—and annealing to  $\sigma_S = 0.3$  removes every one. On the FR3 the same sweep leaves all four configurations at 35/35 strict (17–64 ms per step), since its 2.5 cm tolerance lies above the floor there. The schedule therefore matters exactly when the required precision approaches the floor, which is the regime a surrogate-based planner enters whenever tolerances tighten.

#### F. Rollout Cost and the Error Tube

The planning-time columns quantify the cost decoupling claimed in Sec. I. A sampling planner spends  $S \times N \times T$  model evaluations per control step. For BK-MBD each evaluation is one bilinear matrix–vector recursion in  $\mathbb{R}^r$ , so its per-step cost is set by the lifting dimension alone and is essentially constant across our testbeds (64–95 ms). The oracle’s cost instead tracks the complexity of the true dynamics, from 81 ms on the analytic unicycle to 4.1 s on the physics engine—two orders of magnitude the moment rollouts require a simulator.

The tube cost (15) acts as a soft trust region during the anneal, and Fig. 5 checks empirically that the quantile-fitted recursion still does the job it is asked to do. On 2,000 held-out FR3 rollouts of 40 steps (80,000 propagated steps,  $2.7\times$  the planning horizon), the tube bounds the true lifted error in all but 21 steps (0.026%): the one-step  $1 - \delta = 99.9\%$  coverage of (13) transfers essentially unchanged through the propagation (14). The bound is loose (median  $e_k/\|\delta_k\| \approx 1.8 \times 10^3$  at the planning horizon), which is inherent to the norm recursion: with  $\|A\|_2 \approx 1.00$  and  $\sum_i \|B_i\|_2 \approx 1.24$  the growth factor

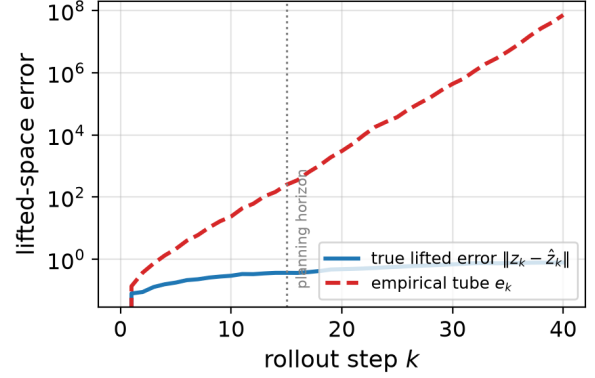


Fig. 5. Empirical tube  $e_k$  vs. the true lifted error on a representative held-out FR3 rollout, with the quantile-fitted constants used by the planner. Over  $2,000 \times 40$  held-out steps the tube is violated in 0.026% of steps, matching the fitted 99.9% one-step coverage.

$\bar{m}(u) + c_x$  exceeds one at every step. For BK-MBD this looseness is immaterial, because the tube enters as a *relative* penalty that orders candidates by how far they leave the trusted region, rather than as a constraint. For the same reason, however, we regard the tube as empirical and attach no certificate semantics to it.

The fitting of the tube, however, must respect the residual distribution. Fitting  $(c_x, c_u)$  to the worst-case residual instead of the quantile bound (13) lets a handful of rest-to-motion transients, which a velocity-free lifted state cannot represent, inflate  $c_x$  by  $40\times$  (from 0.008 to 0.32) on the FR3. The propagated bound (14) then grows exponentially over the horizon, and the tube term—not the task cost—dominates the anneal. With no change to the model, BK-MBD is left oscillating 23 cm from a target that it otherwise reaches with 2.4 cm accuracy. The quantile fit ( $\delta = 10^{-3}$ ) restores the intended regularizing role.

#### G. Delimiting the Regime

The bilinear rollout was motivated by first-order coupling. Two compact studies under the same training recipe locate the boundary of that motivation, in decreasing order of coupling strength. The first study considers a 3D quadrotor in MuJoCo, the canonical platform of the linear-lift control literature, in which the coupling acts one integration below the tracked output: the thrust command must first tilt into an acceleration through the attitude before the position responds. At a single step the bilinear model is far more accurate even on the linear-velocity rows, since the one-step increment  $\Delta v \propto T Re_3$  is a state–input product, and it remains  $2.3\text{--}2.4\times$  more accurate on the full feature vector at  $H = 12$ . Over the horizon, however, both classes are limited by the same attitude-prediction error, and the velocity-row gap settles at  $1.5\text{--}1.8\times$  (three training seeds). The remaining gap reflects *shared* prediction error rather than the directional input-channel bias of Sec. IV-D, so a coupling hidden behind an integrator leaves the linear lift serviceable for position control. This is the very regime in which the Koopman control literature reported its linear-lift

successes (Sec. I). The second study considers a torque-limited pendulum, whose input involves no configuration-dependent gain, so that Proposition 1 imposes no structural penalty: inside the same annealed planner, the linear class with a 128-neuron encoder swings up from hanging on 5/5 planning seeds. The bilinear class contributes parameter economy here rather than feasibility. At the same lifted dimension  $r = 8$ , it matches with a 32-neuron encoder the multi-step accuracy that the linear model needs 256 neurons to reach (mean over three training seeds), and this difference carries into closed loop at tight budgets, where the width-64 linear model fails swing-up (0/5) while the bilinear one already succeeds (4/5). Together with Secs. IV-B–IV-C, these studies delimit the claim of this paper: the bilinear rollout is required when the tracked output is first-order coupled, optional when the coupling acts behind an integrator, and an efficiency improvement when the coupling is absent.

## V. CONCLUSION

This paper presented BK-MBD, which replaces the simulator rollouts of Model-Based Diffusion with a learned bilinear deep Koopman surrogate and a quantile-fitted model-error tube. The motivating representability argument states that the best constant input map is the mean of the configuration-dependent gain, leaving an irreducible error equal to its dispersion. Closed-loop experiments on an SE(2) unicycle, a kinematic 7-DOF arm, and a physics-simulated Franka FR3 confirmed this argument inside a single planning stack. The full-lift linear rollout failed or stalled (0/20, 0/35, and 7/35 strict), while the bilinear rollout matched the true-dynamics oracle in all 90 trials and reduced the FR3 planning step from 4.1 s to 64 ms, within  $1.3\times$  of the 50 ms control period. An optimizer ablation further showed that the annealed schedule itself supplies the final precision that a single-stage sampler cannot reach.

Two limitations qualify these results. First, the tube constants are coverage estimates fitted from data, since certified bounds in the spirit of [21] would require identification machinery that our deep lift does not yet provide. Second, all our tasks, while first-order coupled, are free of contact. Certified tube constants and contact-rich dynamics therefore remain open problems. Conversely, the linear lift remains a serviceable controller under the same planner wherever the input acts state-independently or only through an integrator (Sec. IV-G), so the bilinear class is the repair demanded specifically by first-order coupled outputs. Deployment on the physical FR3 through the same joint-velocity interface is planned as the next step.

## REFERENCES

- [1] H. J. Kappen, “Path integrals and symmetry breaking for optimal control theory,” *J. Stat. Mech.: Theory Exp.*, p. P11011, 2005.
- [2] E. A. Theodorou, J. Buchli, and S. Schaal, “A generalized path integral control approach to reinforcement learning,” *J. Mach. Learn. Res.*, vol. 11, pp. 3137–3181, 2010.
- [3] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1714–1721.
- [4] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2016, pp. 1433–1440.
- [5] C. Pan, Z. Yi, G. Shi, and G. Qu, “Model-based diffusion for trajectory optimization,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, pp. 57 914–57 943, 2024.
- [6] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, “Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4974–4981.
- [7] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [8] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [9] B. O. Koopman, “Hamiltonian systems and transformation in Hilbert space,” *Proc. Natl. Acad. Sci.*, vol. 17, no. 5, pp. 315–318, 1931.
- [10] I. Mezić, “Spectral properties of dynamical systems, model reduction and decompositions,” *Nonlinear Dyn.*, vol. 41, pp. 309–325, 2005.
- [11] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, “A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition,” *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [12] J. L. Proctor, S. L. Brunton, and J. N. Kutz, “Dynamic mode decomposition with control,” *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.
- [13] B. Lusch, J. N. Kutz, and S. L. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” *Nat. Commun.*, vol. 9, p. 4950, 2018.
- [14] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.
- [15] R. Strässer, M. Schaller, K. Worthmann, J. Berberich, and F. Allgöwer, “An overview of Koopman-based control: From error bounds to closed-loop guarantees,” *Annual Reviews in Control*, vol. 61, p. 101035, 2026.
- [16] W. Hao, Y. Fang, Z. Lu, and S. Mou, “Accelerating sampling-based control via learned linear Koopman dynamics,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2026.
- [17] L. C. Jacob, R. Tóth, and M. Schoukens, “Koopman form of nonlinear systems with inputs,” *Automatica*, vol. 162, p. 111525, 2024.
- [18] D. Bruder, X. Fu, and R. Vasudevan, “Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4369–4376, 2021.
- [19] C. Folkestad and J. W. Burdick, “Koopman NMPC: Koopman-based learning and nonlinear model predictive control of control-affine systems,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 7350–7356.
- [20] G. Mamakoukas, M. L. Castano, X. Tan, and T. D. Murphey, “Local Koopman operators for data-driven control of robotic systems,” in *Robotics: Science and Systems (RSS)*, 2019.
- [21] R. Strässer, M. Schaller, K. Worthmann, J. Berberich, and F. Allgöwer, “SafEDMD: A certified learning architecture tailored to data-driven control of nonlinear dynamical systems,” arXiv:2411.10359, 2024.
- [22] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *International Conference on Machine Learning (ICML)*, 2022.
- [23] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [24] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, “Motion planning diffusion: Learning and planning of robot motions with diffusion models,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.